

FILE SYSTEM

32-bit File Allocation Table (FAT 32)

New Technology File System (NTFS)

Sejarah NTFS

Pada awal tahun 90-an, Microsoft memutuskan untuk menciptakan sebuah sistem operasi dengan kualitas tinggi, penampilan menarik, dapat dipercaya dan aman.

Tujuan dari sistem operasi ini adalah sebagai pijakan bagi Microsoft dalam bisnis yang menguntungkan ini dan meraih pangsa pasar yang luas.

Pada waktu itu sistem operasi unggulan Microsoft adalah MS-DOS dan Windows 3.x yang memberikan kekuatan yang diperlukan Microsoft untuk bersaing dengan sistem UNIX.

Namun ada satu kelemahan yang tidak dapat ditutupi, antara lain pada *data storage dan management, sistem jaringan komputer, dan environment yang mendukung.*

Untuk mengatasi kepincangan pada Windows NT yang merupakan produk terbarunya, maka Microsoft menciptakan sebuah file system baru, yaitu *New Technology File System* disingkat *NTFS*.

NTFS diciptakan dengan memadukan suatu konsep file system lain, HPFS (high performance FS), yang digunakan pada OS/2 ditambah teknologi baru hasil temuan Microsoft sendiri.

Keunggulan & Tujuan NTFS

Beberapa tujuan spesifik dari NTFS adalah:

- ❖ **Reliability:** satu hal yang penting dari sebuah file system yang serius adalah bahwa file system tersebut harus dapat pulih kembali dari masalah tanpa kehilangan data hasil. Disini NTFS mencegah hilangnya data dan memperkecil toleransi dari kesalahan dalam *processing*.
- ❖ **Security dan Access Control:** Kelemahan dari FAT adalah ketidakmampuan mengontrol akses file atau folder dari hard disk, sehingga memungkinkan pihak luar untuk mengubah data pada suatu sistem jaringan.

- ❖ **Breaking Size Barriers:** Karena pada sistem FAT dalam hal ini FAT16 tidak dapat mempartisi lebih dari 4GB, sedang NTFS didesain untuk partisi yang jauh lebih besar.
- ❖ **Storage Efficiency:** NTFS lagi-lagi memperbaiki kelemahan pada FAT16 karena pada sistem ini memungkinkan terjadinya ketidakefisienan pada penyimpanan pada kapasitas hard disk. Untuk itu NTFS menggunakan metode lain dalam alokasi kapasitas hard disk tersebut.
- ❖ **Long File Names:** NTFS memungkinkan nama sebuah file hingga 255 karakter, dibandingkan dengan pada FAT adalah 8+3 karakter.

- ❖ **Networking:** saat ini networking berkembang pesat dengan NTFS memungkinkan networking dalam skala besar.
- ❖ **Storage Fault Tolerance:** Data-redundant storage methods dapat diterapkan pada NTFS. Hal ini berguna dalam menjamin dan melindungi jika suatu data/berkas mengalami kerusakan dengan mengkopi ulang data yang sama dari disk mirror.
- ❖ **Multiple Data Stream:** NTFS dapat terdiri dari lebih 1 stream. Stream tambahan ini dapat berisi berbagai jenis data, walau data itu hanya mendeskripsikan berkas atau **metadata**.

- ❖ **Unicode Names:** Unicode merupakan paket karakter standar yang digunakan pada NTFS dan menggantikan karakter older-single byte ASCII. Setiap karakter pada kebanyakan bahasa yang natural adalah direpresentasikan dengan double-byte number dalam paket karakter Unicode.
- ❖ **Improved File Attribute Indexing:** Dalam NTFS juga terdapat kemampuan untuk memberi indeks pada atribut berkas, fungsinya ialah sebagai penglokasian dan sorting.
- ❖ **Data Compression:** Dalam kompresi data metode yang digunakan adalah Lempel- Ziv Compression. Dengan algoritma ini dipastikan tidak ada data yang hilang pada proses kompresi.

- ❖ **Encryption:** NTFS juga menyediakan Encrypted File System atau EFS untuk perlindungan *cryptografic* pada berkas atau direktori.
- ❖ **Reparse Points:** Dalam NTFS, sebuah berkas atau direktori dapat berisi *reparse point*, dimana terdapat sekumpulan dari user-defined data.

- NTFS menggunakan atribut untuk menyimpan informasi-informasi dari semua file dan direktori. Dalam disk atribut ini dibagi menjadi dua komponen : header dan data. Header menyimpan tipe atribut, nama dan *flag*, serta mengidentifikasikan lokasi dari data atribut.
- NTFS mempunyai 14 tipe atribut sebagai berikut :

Attribute Type	Description
\$VOLUME_VERSION	Volume version
\$VOLUME_NAME	Disk's volume name
\$VOLUME_INFORMATION	NTFS version and dirty flag
\$FILE_NAME	File or directory name
\$STANDARD_INFORMATION	File time stamps and hidden, system, and read-only flags
\$SECURITY_DESCRIPTOR	Security information
\$DATA	File data
\$INDEX_ROOT	Directory content
\$INDEX_ALLOCATION	Directory content
\$BITMAP	Directory content mapping
\$ATTRIBUTE_LIST headers	Describes nonresident attribute
\$SYMBOLIC_LINK	Unused
\$EA_INFORMATION	OS/2-compatibility extended attributes
\$EA	OS/2-compatibility extended attributes

- NTFS menggunakan log file untuk mencatat semua informasi *redo* dan *undo* dari suatu transaction. NTFS menggunakan informasi redo untuk mengulangi transaction. Informasi undo memungkinkan NTFS untuk membatalkan transaction yang belum selesai atau salah.
- Cluster Remapping
 - Ketika terjadi *bad-sector error*, NTFS melakukan pemulihan dengan menggunakan teknik *cluster remapping*. Ketika sistem operasi mendeteksi suatu *bad-sector*, NTFS secara dinamis memetakan ulang cluster yang mengandung bad sector dan mengalokasikan cluster baru untuk data dalam bad sector tersebut.
 - Jika error terjadi pada saat pembacaan data, NTFS mengembalikan pesan *read error* kepada program pemanggil.

- Jika error terjadi pada saat proses menulis, NTFS menulis data pada cluster yang baru, sehingga tidak ada data yang hilang.
- NTFS menyimpan alamat dari cluster yang mengandung *bad sector* dalam *bad cluster file* sehingga bad sector tidak digunakan kembali.

Second Extended File System (Ext2 FS)

Latar belakang

- Second Extended File system (Ext2) dirancang oleh Rémy Card, sebagai file sistem yang extensible dan powerful untuk digunakan pada sistem operasi Linux.
- Ext2 merupakan File System yang paling banyak digunakan oleh para pengguna Linux
- Ext2 pertama kali dikembangkan dan diintegrasikan pada kernel Linux, dan sekarang ini sedang dikembangkan juga penggunaannya pada sistem operasi lainnya.

- ❑ Tujuannya adalah untuk membuat suatu file system yang powerful, yang dapat mengimplementasikan file-file semantik dari UNIX dan mempunyai pelayanan advance features.

Konsep File dalam Unix

- Dalam sistem Unix File merupakan kumpulan data yang saling berhubungan, yang disimpan pada lokasi tertentu dalam disk.
- Inode adalah Informasi yang mengidentifikasikan suatu file secara unik. Inode mengidentifikasikan lokasi tempat file disimpan, dan karakteristik dari file tersebut. (owner, date, dsb); tetapi nama file tidak disimpan sebagai bagian dari inode. Informasi yang disimpan dalam inode antara lain :
 - Lokasi blok data suatu file dalam disk.
 - Ukuran file (jumlah bytes)
 - Waktu dan tanggal pembuatan/modifikasi/akses dari suatu file
 - user dan group ownership
 - Permission settings

❏ Nama tidak disimpan sebagai bagian dari file. File tidak mempunyai informasi tentang namanya sendiri. Nama merupakan suatu reference (link) ke suatu inode. Nama dari file disimpan dalam suatu file khusus yang disebut **direktori**.

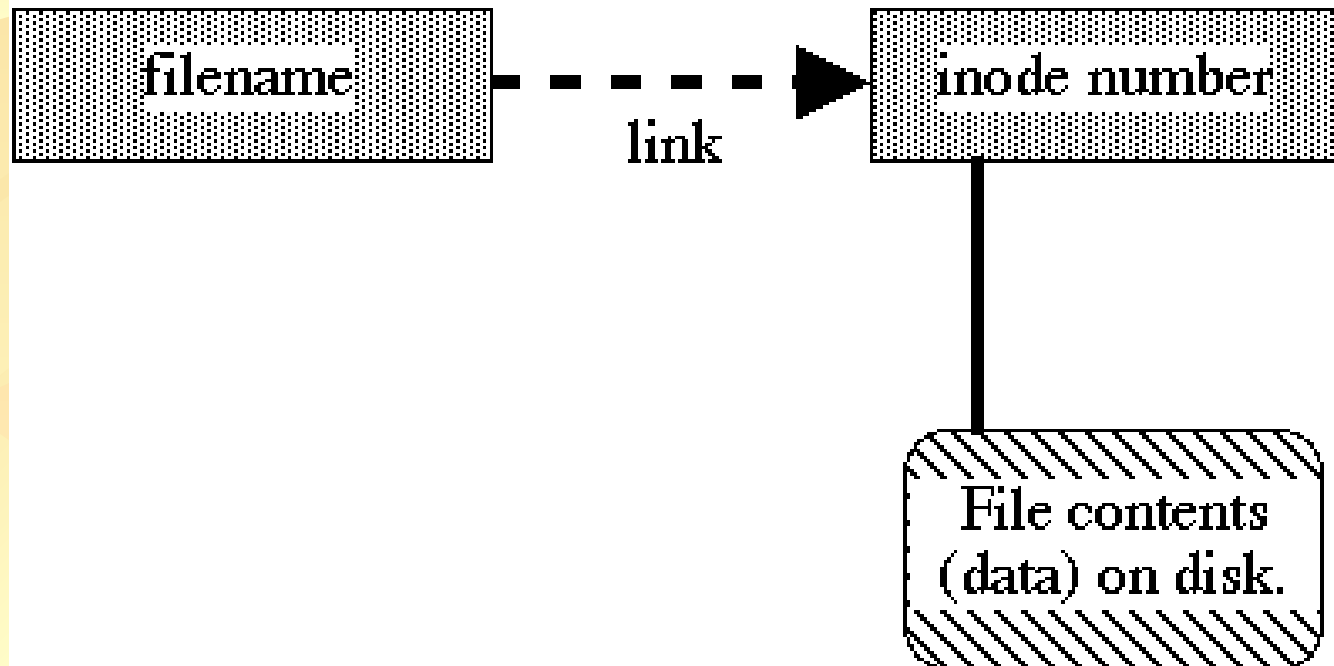
❏ Teknik penyimpanan nama file ini memberikan beberapa keuntungan, yaitu :

- Memungkinkan struktur penyimpanan secara hierarkis
- Memperbolehkan multiple links (name) ke satu file yang sama
- “Regular links” harus menunjuk pada file yang berada pada direktori yang sama, dan tidak dapat menunjuk ke direktori lain.
- "Symbolic links" dapat menunjuk pada file yang berada pada direktori lain, dan dapat menunjuk ke direktori lain.

How UNIX Files Are Viewed

As you see it:

As the kernel (UNIX) sees it:

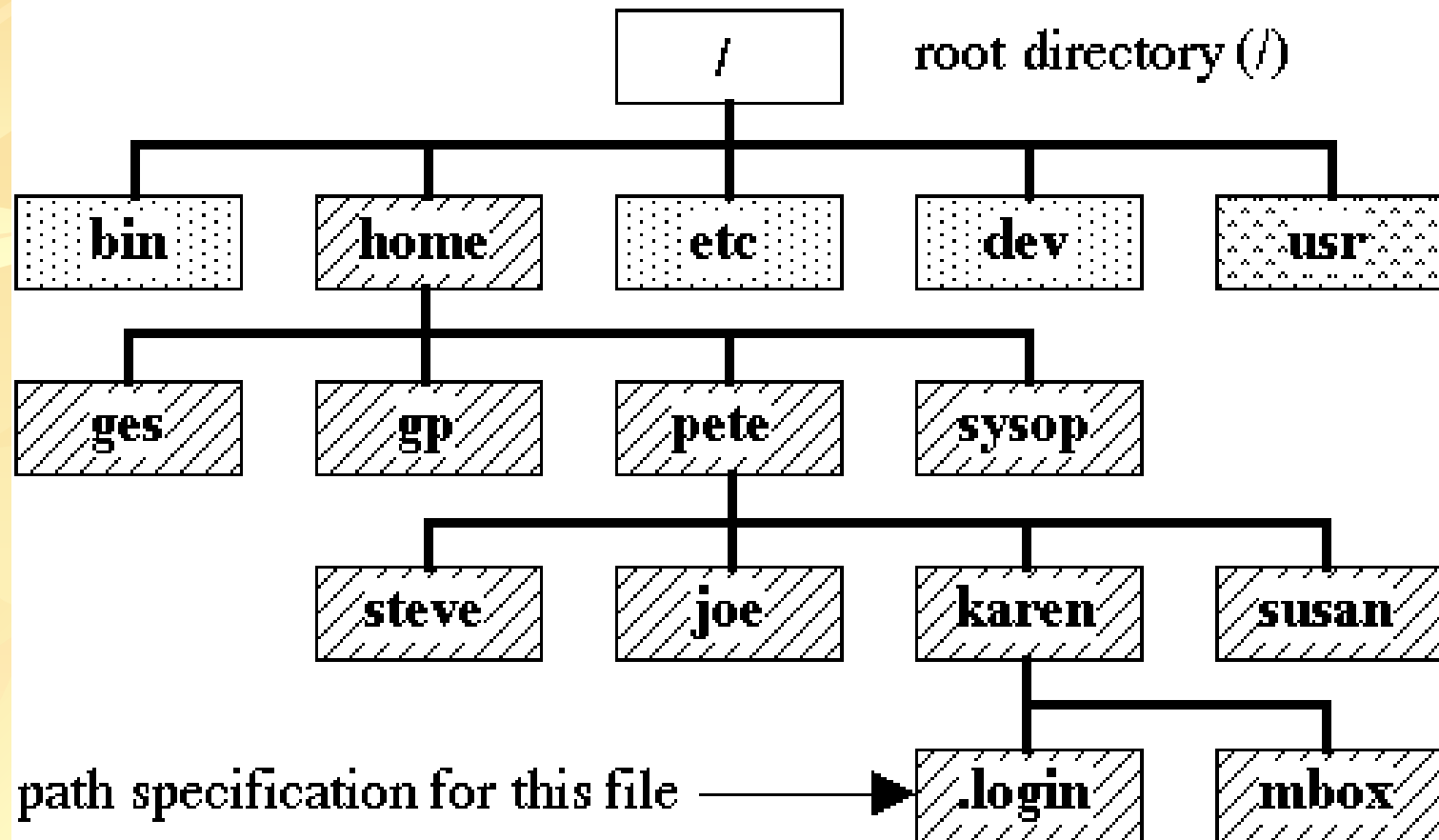


Konsep Direktori dalam Unix

- ➔ Direktori adalah file khusus yang mengandung daftar nama-nama dari file-file lainnya dan inode yang bersesuaian. Suatu direktori dapat menunjuk ke inode yang sama dengan menggunakan multiple links.
- ➔ Dua nama khusus terdapat dalam setiap direktori, yaitu “.” (titik) dan “..”(titik titik). Seperti halnya file lainnya, direktori tidak mempunyai informasi tentang namanya sendiri, sehingga menggunakan nama simbolis “.” untuk menunjuk ke direktori itu sendiri. Simbol “..” menunjuk ke parent dari direktori tersebut.
- ➔ Direktori dapat mengandung nama-nama dari direktori-direktori lainnya. Hal ini menciptakan struktur tree

- ➔ Puncak dari semua direktori adalah direktori “root” . Karena file tidak mengetahui namanya sendiri, maka direktori root tidak mempunyai nama tetapi direpresentasikan dengan single slash (/).
- ➔ Ketika menunjuk kepada suatu file yang spesifik atau direktori pada tree, digunakan konsep “pathname”: urutan direktori, dimulai dari root, setiap direktori dipisahkan dengan slashes (/).
- ➔ Struktur tree file system dapat terdiri dari beberapa disk, yang terlihat seolah-olah satu disk besar. Setiap satuan disk itu sebenarnya memiliki struktur treenya masing-masing dengan direktori root masing-masing. Root dari suatu disk diasosiasikan (mounted) oleh operating system dengan nama direktori pada disk lainnya untuk menghubungkan disk secara hierarkis.

UNIX File System Hierarchy (sample)



`/home/pete/karen/.login`

Kemampuan dasar EXT2

- ➔ File system EXT2 mampu menyokong beberapa tipe file yang standar dari UNIX, seperti regular file, directories, device special files, dan symbolic links.
- ➔ EXT2 mampu mengatur file-file system yang dibuat dalam partisi yang besar.
- ➔ File system EXT2 mampu menghasilkan nama-nama file yang panjang. Maximum 255 karakter.
- ➔ EXT2 memerlukan beberapa blok untuk super user (root).

The background of the slide features a repeating pattern of stylized, overlapping leaves in various shades of yellow and orange, creating a textured, organic feel.

Third Extended File System (Ext3)

- EXT3 merupakan suatu journalled filesystem
- Journalled filesystem didesain untuk membantu melindungi data yang ada di dalamnya.
- Dengan adanya journalled filesystem, maka kita tidak perlu lagi untuk melakukan pengecekan kekonsistensian data, yang akan memakan waktu sangat lama bagi harddisk yang berkapasitas besar.
- EXT3 adalah suatu filesystem yang dikembangkan untuk digunakan pada sistem operasi Linux.
- EXT3 merupakan hasil perbaikan dari EXT2 ke dalam bentuk EXT3 yang lebih baik dengan menambahkan berbagai macam keunggulan.

Keunggulannya:

■ Availability :

- ∂ EXT3 tidak mendukung proses pengecekan file system, bahkan ketika system yang belum dibersihkan mengalami “shutdown”, kecuali pada beberapa kesalahan hardware yang sangat jarang.
- ∂ Hal seperti ini terjadi karena data ditulis atau disimpan ke dalam disk dalam suatu cara sehingga file system-nya selalu konsisten.
- ∂ Waktu yang diperlukan untuk me-recover ext3 file system setelah system yang belum dibersihkan dimatikan tidak tergantung dari ukuran file system atau jumlah file; tetapi tergantung kepada ukuran “jurnal” yang digunakan untuk memelihara konsistensi. Jurnal dengan ukuran awal(default) membutuhkan sekitar 1 sekon untuk recover (tergantung dari kecepatan hardware).

■ Integritas Data

- ✗ Dengan menggunakan file sistem ext3 kita bisa mendapatkan jaminan yang lebih kuat mengenai integritas data dalam kasus dimana sistem yang belum dibersihkan dimatikan (shutdown).
- ✗ Kita bisa memilih tipe dan level proteksi yang diterima data. Kita bisa memilih untuk menjaga agar file system tetap konsisten, tetapi tetap mengizinkan kerusakan terhadap data dalam file system dalam kasus dimatikannya (shutdown) system yang belum dibersihkan; ini bisa memberikan peningkatan kecepatan pada beberapa keadaan.

- ✗ Secara alternatif kita bisa memilih untuk lebih memastikan bahwa data konsisten dengan bagian dari file system; ini berarti kita tidak akan pernah melihat “garbage data” pada file-file yang baru ditulis ulang setelah terjadi “crash”.
- ✗ Pilihan yang aman yakni menjaga kekonsistenan data sebagai bagian dari file system adalah pilihan default

■ Kecepatan

Meskipun melakukan penulisan suatu data lebih dari satu kali, ext3 seringkali lebih cepat daripada ext2 karena sistem penjurnalan ext3 mengoptimasi pergerakan head “hard drive”.

Kita bisa memilih dari tiga mode penjurnalan untuk mengoptimasi kecepatan, memilih untuk melakukan “trade off” terhadap integritas data, yaitu:

- γ Mode pertama, data = writeback, membatasi jaminan integritas data, memperbolehkan data yang lama untuk muncul dalam file-file setelah terjadi “crash”

- Mode kedua, data = ordered(mode default), menjamin bahwa data konsisten dengan file system; file-file yang baru ditulis tidak akan pernah menjadi “garbage contents” setelah terjadi crash.
- Mode terakhir, data = journal, membutuhkan jurnal yang lebih besar untuk kecepatan yang wajar dalam sebagian besar kasus dan oleh karena itu membutuhkan waktu lebih banyak untuk “recover” dalam kasus dimatikannya system yang belum dibersihkan (unclean shutdown), tetapi terkadang lebih cepat untuk operasi-operasi basis data tertentu.

■ Easy Transition

Adalah hal yang mudah untuk mengubah dari ext2 menjadi ext3 dan mendapatkan keuntungan-keuntungan dari sistem penjurnalan yang kuat, tanpa harus format ulang. Ada dua cara untuk melakukan transisi :

- Program instalasi Red Hat Linux menawarkan untuk mentransisi file system yang ada ketika kita meng-upgrade sistem. Yang perlu dilakukan adalah memilih satu “checkbox” setiap file sistem.
- Program tune2fs bisa menambahkan jurnal ke file system ext2 yang sudah ada. Bila file system sudah di-mount ketika ditransisi, maka jurnal akan bisa dilihat seperti file. Sebaliknya kalau belum di-mount jurnal akan tersembunyi dan tidak akan muncul di file system. Hanya dengan menjalankan **tune2fs -j /dev/hdal** dan merubah ext2 menjadi ext3 **/etc/fstab**.

- Ext3 menyediakan tiga mode journaling sebagai berikut :
 - journal - mencatat semua perubahan pada file system dan metadata. Paling lambat diantara ketiga mode journaling Ext3. Mode ini meminimalkan kemungkinan hilangnya perubahan yang terjadi pada setiap file dalam file system Ext3.
 - ordered - hanya mencatat perubahan yang terjadi pada metadata, dan membiarkan data pada disk diupdate sebelum perubahan pada metadata dari file tersebut dilakukan. Mode journaling ini adalah mode journaling default yang digunakan oleh Ext3.
 - writeback - hanya mencatat perubahan pada metadata, tetapi bergantung pada proses penulisan standar dari file system untuk menulis perubahan data dari file kedalam disk. Mode ini adalah mode journaling tercepat yang dimiliki Ext3.

- Menggunakan mode “journal” berarti bahwa Ext3 harus mencatat setiap perubahan pada file system sebanyak dua kali, pada journal dan pada file system itu sendiri. Hal ini dapat mengurangi kinerja dari file system secara keseluruhan. Namun mode ini meminimalkan kemungkinan hilangnya perubahan yang akan dilakukan pada file system, karena update pada metadata maupun update dari file data disimpan dalam journal dan dapat dilakukan kembali setelah sistem di-reboot.

- Menggunakan mode “ordered” berarti hanya perubahan pada metadata yang dicatat. Hal ini mengurangi pengulangan penulisan pada file system dan journal, sehingga mode ini lebih cepat. Karena perubahan pada file data tidak dicatat maka perubahan ini harus dilaksanakan sebelum perubahan pada metadata yang bersesuaian dilakukan.
- Menggunakan mode “writeback” lebih cepat daripada dua mode lainnya karena mode ini hanya mencatat perubahan pada metadata dan tidak menunggu perubahan pada file data yang bersangkutan dilakukan. Karena update pada file data dilakukan secara asinkronus dengan perubahan pada metadata, file dalam file system dapat mengalami inkonsistensi metadata, namun hal ini tidaklah fatal.

Mengapa kita bisa mempercayai EXT3?

Di bawah ini akan dituliskan beberapa hal yang telah dilakukan oleh EXT3 (berdasarkan penelitian yang dilakukan oleh Michael K. Johnson dkk.) yang dapat meyakinkan bahwa EXT3 aman untuk menangani data-data dari pemakai:

- Michael K. Johnson dkk. telah memeriksa EXT3 terhadap berbagai macam kondisi, termasuk kesalahan alokasi memori yang dapat terjadi kapan saja. Mereka telah mengujinya berulang-ulang dan seringkali (setiap saat kodenya diubah) dengan memaksa *error* dan lalu memeriksa kekonsistensian *file system*-nya.

- ❁ Michael K. Johnson dkk. telah memeriksa dan menguji EXT3 dalam hal interaksi yang buruk dengan VM *subsystem*, mereka mencari dan memperbaiki beberapa interaksi. Sebuah *journaling file system* meletakkan banyak tekanan pada VM *subsystem* dan mereka telah menemukan dan memperbaiki *bugs* pada EXT3 dan VM *subsystem* pada proses pemeriksaan dan pengujian ini. Setelah melewati ribuan jam untuk proses pemeriksaan dan pengujian ini, mereka akhirnya amat sangat percaya terhadap sifat *robustness* dari *file system* EXT3 ini.
- ❁ Michael K. Johnson dkk. telah melakukan uji tekanan di bawah kumpulan konfigurasi-konfigurasi yang besar. Pekerjaan ini telah menghabiskan ribuan jam untuk merencanakan *load testing* pada banyak variasi *hardware* dan konfigurasi *file system*, sebagaimana banyaknya kasus yang akan diuji.

- ❁ Michael K. Johnson dkk. telah mengatur untuk memperbolehkan *user* memilih untuk memeriksa kekonsistensian *file system* setelah terjadi *unclean system shutdown*, meskipun *file system*-nya telah ditandai “clean”, dengan alasan untuk menghadapi kemungkinan *hardware* menyebabkan *corruption*.

Hal ini disebabkan oleh gangguan atau kerusakan *hardware*, dan khususnya adalah gangguan sumber daya listrik, dapat menyebabkan data “sampah” ditulis hampir pada semua tempat di disk.

Menekan tombol *reset* tampaknya bukanlah pemicu dari masalah ini, tetapi gangguan sumber daya yang dapat diasosiasikan dengan kejadian seperti sambaran kilat dan pohon yang jatuh menimpa kabel tiang listrik lebih cenderung mengakibatkan kerusakan yang dapat merusak data dan jalur penyimpanan menuju disk.

IDE *system* cenderung sedikit lebih rentan terhadap masalah ini daripada SCSI *system* dalam bagian karena disk IDE lebih cenderung mengimplementasikan *looser caching algorithms*.

- *Feature* ini diimplementasikan dengan menggunakan file ***/.autofsck***. Jika *root user* menghapus file tersebut selama operasi normal, maka sistem akan menawarkan untuk memeriksa konsistensi *file system* pada waktu *boot*. Jika ***/.autofsck*** hilang dan *user* memilih untuk memaksa pemeriksaan konsistensi *file system*, maka efeknya akan sama seperti apabila file ***/forcefsck*** ada.



Perbandingan antar File System (FS)

Perbandingan FAT32 VS NTFS

- ★ Berikut ini akan dibandingkan kinerja dan kemampuan dari FAT32 dengan NTFS.
- ★ Kriteria-kriteria yang menjadi dasar perbandingan kedua file system ini antara lain :
 - ★ Kecepatan akses
 - ★ Ukuran partisi dan banyak file
 - ★ Keamanan data
 - ★ daya tahan
 - ★ Efisiensi penggunaan *disk space*
 - ★ Kecocokan dengan file system lainnya.

★ **Kecepatan akses**

- ★ NTFS dapat mengakses file dengan cepat karena NTFS menyimpan data atribut dalam MFT, namun jika file terfragmentasi menjadi banyak bagian, maka perpindahan head dari disk akan memperlambat pengaksesan.
- ★ Pada FAT32, proses pengaksesan file akan jauh lebih lambat jika file terfragmentasi dan bagian-bagiannya tersebar berjauhan dalam disk.
- ★ Pada NTFS satu cluster direpresentasikan dengan 1 bit, sehingga NTFS membutuhkan waktu yang jauh lebih sedikit untuk menemukan free space dibandingkan dengan FAT32.

- ★ Dalam pencarian *free cluster*, FAT melakukan pemeriksaan pada tabel FAT untuk menemukan free cluster.
- ★ Secara umum NTFS mempunyai kecepatan akses file yang lebih baik daripada FAT32.

★ Ukuran partisi dan banyak file

- ★ Baik NTFS maupun FAT32 mempunyai ukuran maksimum partisi sebesar 2 terabytes.
- ★ NTFS memiliki ukuran maksimum file yang hampir tidak terbatas, terbatas sampai sebesar ukuran partisi. Sedangkan FAT32 mempunyai ukuran maksimum file sebesar 4 gigabytes.
- ★ NTFS dan FAT32 mempunyai jumlah maksimum file yang hampir tidak terbatas.

- ★ FAT32 mempunyai batas maksimum jumlah cluster sebanyak 268.435.456 cluster dalam satu partisi, sedangkan NTFS jumlah cluster maksimumnya hampir tidak terbatas.
- ★ Struktur direktori NTFS lebih efektif daripada FAT32. jika dalam suatu direktori terdapat ribuan file maka kecepatan NTFS jauh lebih cepat daripada FAT32.

★ Keamanan data

- ★ NTFS memiliki *built-in security*, yang memungkinkan untuk mengatur *permission* dari masing-masing file maupun direktori. Hal ini merupakan salah satu keunggulan NTFS yang tidak dimiliki FAT32.
- ★ keunggulan ini menyebabkan NTFS sangat baik untuk komputer yang berbasis *network*.

★ Daya tahan

- ★ NTFS menggunakan *standart transaction logging*, sehingga pemulihan terhadap kesalahan yang tidak diinginkan dapat dilakukan dengan mudah.
- ★ FAT32 tidak memiliki metode untuk melakukan perbaikan dan pemulihan data, sehingga sistem ini rentan terhadap kerusakan.
- ★ NTFS lebih unggul dari FAT32 dalam hal daya tahan dan keamanan data.

★ Efisiensi penggunaan disk space

- ★ Penggunaan *disk space* pada NTFS lebih hemat daripada FAT32 karena NTFS menggunakan ukuran cluster yang lebih kecil.
- ★ Ukuran cluster yang kecil membuat disk space yang terbuang lebih sedikit.

★ kecocokan dengan file system lain

- ★ Partisi yang menggunakan NTFS hanya dapat diakses oleh partisi lain yang juga menggunakan NTFS. Partisi yang menggunakan NTFS tidak dapat dibaca oleh partisi yang menggunakan FAT.
- ★ Partisi yang menggunakan FAT32 dapat diakses oleh partisi lain baik yang menggunakan FAT32 maupun NTFS.
- ★ FAT32 lebih compatible, baik dengan FAT32 sendiri maupun NTFS.

Table perbandingan FAT32 VS NTFS

Kriteria	FAT32	NTFS
Ukuran maksimum partisi	2 TB	2 TB
Jumlah maksimum cluster	268.435.456	Hampir tak terbatas
Jumlah file maksimum	Hampir tak terbatas	Hampir tak terbatas
Ukuran file maksimum	Hanya dibatasi oleh ukuran patrisi	4 GB
Nama file	Maksimum 255 karakter	Maksimum 255 karakter
Security	tidak memiliki fasilitas file security	Memiliki <i>file permission</i>

Lokasi boot sector	Sector pertama	Sector pertama dan terakhir
System Records Mirror	Second copy of FAT	MFT Mirror File
Daya tahan dan pemulihan data	Rendah	Daya tahan tinggi, karena menggunakan <i>standart transaction logging</i>
Efisiensi penggunaan disk space	Cukup baik	Sangat baik
Kinerja	Baik untuk ukuran partisi yang kecil	Baik untuk ukuran partisi yang besar

Perbandingan EXT2 VS EXT3

- ⌘ Secara umum prinsip-prinsip dalam EXT2 sama dengan EXT3. Metode pengaksesan file, keamanan data, dan penggunaan *disk space* antara kedua file system ini hampir sama.
- Perbedaan mendasar antara kedua file system ini adalah konsep *journaling file system* yang digunakan pada EXT3.
- Konsep journaling ini menyebabkan EXT2 dan EXT3 memiliki perbedaan dalam hal daya tahan dan pemulihan data dari kerusakan.
- Konsep journaling ini menyebabkan EXT3 jauh lebih cepat daripada EXT2 dalam melakukan pemulihan data akibat terjadinya kerusakan.